
# Upgrade Reference

Version 2026.1
2026-04-20

InterSystems Corporation One Congress Street Boston MA 02114 www.intersystems.com

*Upgrade Reference*
PDF generated on 2026-04-20
InterSystems IRIS Version 2026.1


For Support questions about any InterSystems products, contact:

**InterSystems Worldwide Response Center (WRC)**
Tel: +1-617-621-0700
Tel: +44 (0) 844 854 2917
Email: support@InterSystems.com

# Table of Contents

# 1

# Special Considerations When Upgrading

This page includes special considerations that should be reviewed when upgrading. Each item includes *Affects Upgrade From Versions* (upgrades *from* these versions are impacted), the *Conditions* where the item is relevant, the *Ramifications* of the item, and any *Instructions* that should be followed for addressing the item. These instructions supplement Pre-Upgrade Steps.

The considerations are listed from most recent to oldest affected versions. You should review items as far back as the release you are upgrading from before starting your upgrade.

**Note:** A special upgrade procedure is required when upgrading from Health Connect 15.03. For details, see the *InterSystems IRIS Migration Guide*, which is available from the InterSystems WRC Documents page.

## 1.1 Allow $increment with Shared Block Ownership

### Affects Upgrades From Versions:

All versions.

### Conditions:

This release allows **$INCREMENT**, in most cases, to function internally with only shared ownership of a data block. This enables concurrent reads of data stored in the same block and concurrent executions of **$INCREMENT** on global nodes stored in the same block (performed using compare-and-swap within the block). The semantics of **$INCREMENT** to application code remain unchanged.

Journal records for **$INCREMENT** operations that were performed with shared ownership may now appear out of order in the journal file (for example, the journal record for a **$INCREMENT** that set ^X to 100 may appear before the one setting it to 99). All **$INCREMENT** journal records still use the SET type, but now also feature an extended type that is either `SET ($I)` or `SET ($I if greater)`. The latter value represents a **$INCREMENT** operation that may appear out of order and is only applied by a journal restore if the value captured in the journal record is greater than the current value of the current value of the global node. Note that the journal record a **$INCREMENT** results in is unpredictable, as the ability to use shared ownership is dependent on some internal conditions. **$INCREMENT** using a negative increment value is always done with exclusive access and is journaled with `SET ($I)` as its extended type.

### Ramifications:

Any custom logic that takes action using the value from a SET record may need to disambiguate between the new extended types, but only if the logic is sensitive to the order in which **$INCREMENT** records are encountered.

### Instructions:

Read the journal file using SYS.Journal.Record objects to access the new ExtType and ExtTypeName properties.

Any custom logic that access journal records with macros in %syJrnRecord.inc can access the extended types, `SET ($I)` or `SET ($I if greater)`, with $$$JRNTYPE1, which returns $$$JRNINCRYP or $$$JRNINCRIGTYP for **$INCREMENT** operations.

# 1.2 InterSystems IRIS BI Builds and Synchronizes Dependent Cubes Automatically

### Affects Upgrades From Versions:

All versions.

### Conditions:

Instances which build or synchronize InterSystems IRIS Business Intelligence cubes as part of a custom task or method.

### Ramifications:

When you invoke %BuildCube() or %SynchronizeCube() on any cube which has a relationship, the methods collate a list of all cubes which depend upon the specified cube. The methods then perform their respective update operations upon the dependent cubes in addition to the specified cube, determining the correct update sequence automatically. (Internally, the methods now invoke **%BuildOneCube()** and **%SynchronizeOneCube()** to update an individual cube.)

Because of this, custom tasks or methods which invoke these methods to build or synchronize related cubes one at a time may perform unnecessary, duplicate work upon upgrade.

### Instructions:

To permanently address this:

1. Review your related cubes. Note which cubes depend upon others and which do not. (Dependent cubes *do* define a source expression for a relationship; independent cubes do *not*.) Refer to the output of the %DeepSee.CubeUtils class's %GetCubeGroups() method for guidance: if a cube has any cubes which depend upon it, the output *pCubes* includes a *pCubes(<cubeKey>, "dependents")* array. This array identifies the dependent cubes which Business Intelligence will update automatically after the cube identified by *<cubeKey>* has been updated.
2. Edit your custom update tasks and methods. Remove any calls to %BuildCube() or %SynchronizeCube() for a dependent cube which immediately follow a call to %BuildCube() or %SynchronizeCube() on its independent counterpart.

# 1.3 MPRLLIB Database

### Affects Upgrades From Versions:

All versions.

### Conditions:

HealthShare Health Connect and InterSystems IRIS for Health users who previously migrated from Health Connect/HSAP based on the Caché/Ensemble platform.

### Ramifications:

The components.ini file in your installation directory may have a reference to the database MPRLLIB, which is no longer used by the product. This causes a misleading error message in messages.log saying that this database does not exist. This will prevent a misleading error message in messages.log saying that this database does not exist.

### Instructions:

Comment out the reference to this database prior to the upgrade by inserting a semicolon at the start of each line.

Example:

```
;[MPRLLIB]
;Version=15.032.9686
[HSLIB]
Version=2018.1.0
Compatibility_HSAALIB=15.0
Compatibility_HSPILIB=14.0
Compatibility_VIEWERLIB=17.0
```

# 1.4 Purge Incomplete Business Process Logs Before Upgrading

### Affects Upgrades From Versions:

2023.2 and earlier

### Conditions:

Instances with large numbers of incomplete business process logs. View your business process instances and evaluate if you have large numbers of incomplete business processes.

### Ramifications:

Since incomplete business process logs are not automatically purged, your instance may have large numbers of incomplete business processes. During the upgrade process, the storage for these logs is upgraded. Large numbers of incomplete business processes can slow your upgrade and cause it to fail.

### Instructions:

To address this problem, *before* upgrading, purge incomplete business processes. Make sure you disable the `Purge only completed session (KeepIntegrity)` setting so that *incomplete* business processes are also purged. You can purge business processes using the Message Purge API by running a command like the following (set `pDaysToKeep` to an integer):

```
Set tSC=##class(Ens.BusinessProcess).Purge(.tDeletedCount,pDaysToKeep,0)
```

# 1.5 New “Configure Secure Communication” Option in the Installer Wizard

### Affects Upgrade From Versions:

2022.1 and earlier.

### Conditions:

All environments. Environments with active configuration in the **Configure SSL Access** dialog must perform additional steps. Note that the **Configure SSL Access** dialog in the Installer Wizard has been renamed to **Configure Secure Communication**.

### Ramifications:

In the new dialog you must now specify an SSL/TLS Configuration in order to make the secure communication settings Active. A default value of `HS.Secure.Demo` is entered for the SSL/TLS Configuration setting upon upgrade.

### Instructions:

If you previously had an Active configuration in the **Configure SSL Access** option, you must modify the default value to reflect the SSL/TLS configuration in use on your instance as follows:

1. Navigate to the Installer Wizard.
2. Select the new **Configure Secure Communication** option.
3. Confirm that your **Secure Port** is identified and the **These Settings are Active** checkbox is selected.
4. In the **SSL/TLS Configuration** field, select the name of the SSL/TLS configuration in use on your instance.
5. Click **Save**.

1. Navigate to the Installer Wizard.
2. Select the new **Configure Secure Communication** option.
3. Confirm that your **Secure Port** is identified and the **These Settings are Active** checkbox is selected.
4. In the **SSL/TLS Configuration** field, select the name of the SSL/TLS configuration in use on your instance.
5. Click **Save**.

# 1.6 Recompile Classes Featuring Properties with MAXLEN=""

### Affects Upgrade From Versions:

2022.1.1 and earlier.

**Conditions:**

Environments with existing classes containing a property with MAXLEN="".

**Ramifications:**

If an existing class contains a property with MAXLEN="", SQL queries on tables based on that class return an error after upgrading.

**Instructions:**

Recompile the affected classes.

# 1.7 External Language Gateway Configurations

**Affects Upgrade From Versions:**

2022.1.1 — 2021.1.0

**Conditions:**

Environments where all external language gateway configurations have been removed.

**Ramifications:**

You may encounter validation errors during the upgrade process.

**Instructions:**

To prevent these errors, add a single gateway configuration of type **Remote** that points to the local gateway with an arbitrary port number. For example, you can set the **Server Name / IP Address** to `127.0.0.1` and set the **Port** to `1`, naming it `ForUpgrade`. This can be done at any point prior to upgrading and will not impact normal system operation. This configuration can be deleted after the upgrade is completed.

# 1.8 C Runtime Requirements on AIX®

**Affects Upgrade From Versions:**

2021.2 and earlier.

**Conditions:**

Environments running on AIX®.

**Ramifications:**

Health Connect on AIX is compiled using the IBM XL C/C++ for AIX 16.1.0 compiler.

**Instructions:**

If the system on which you are installing HealthShare Health Connect does not have the corresponding version of the runtime already installed, you must install it.

# 1.9 Updating FHIR Endpoints

### Affects Upgrade From Versions:

2020.3 and earlier.

### Conditions:

The following steps that may be required depending upon on how you have customized your FHIR server. Perform these tasks in the following order:

1. If your FHIR server uses custom subclasses, you must modify your architecture subclasses.
2. If your FHIR endpoint uses custom search parameters, migrate them to a FHIR package and apply them to the endpoint.

### Ramifications:

InterSystems IRIS for Health may not have access to your modifications, which could cause errors.

### Instructions:

*Step 1: Modifying Architecture Subclasses*

As part of the FHIR architecture that was introduced in InterSystems IRIS for Health 2020.1, you can use a custom InteractionsStrategy to implement a custom FHIR server. If your FHIR server's endpoint uses a custom InteractionsStrategy, including if it uses a subclass of the Resource Repository, complete the following steps:

1. Complete the upgrade of your InterSystems IRIS for Health instance.
2. Using your IDE, do *one* of the following in your endpoint's namespace:
   - If the InteractionsStrategy of your endpoint extended the Resource Repository (`HS.FHIRServer.Storage.Json.InteractionsStrategy`), create a subclass of `HS.FHIRServer.Storage.Json.RepoManager`.
   - If the InteractionsStrategy of your endpoint subclassed `HS.FHIRServer.API.InteractionsStrategy` directly, create a subclass of the `HS.FHIRServer.API.RepoManager` superclass.
3. Add the following parameters to your subclass of the Repo Manager:
   - `StrategyClass` — Specifies the subclass of your InteractionsStrategy.
   - `StrategyKey` — Specifies the unique identifier of the InteractionsStrategy. This must match the value of the `StrategyKey` parameter in the InteractionsStrategy subclass.
4. If your InteractionsStrategy subclass included custom code for the methods that manage the Service, you must move that logic to the new methods in the Repo Manager subclass that you created. Specifically, you must move custom code from the `Create`, `Delete`, `Decommission`, and `Update` methods to the corresponding methods in your Repo Manager subclass (`CreateService`, `DeleteService`, `DecommissionService`, and `UpdateService` ).

*Step 2: Migrating Custom Search Parameters to a FHIR Package*

In versions of IRIS for Health before 2020.4, using custom FHIR search parameters required you to define a custom metadata set. In this version, defining FHIR metadata, including custom search parameters, has been migrated to FHIR packages. When you upgrade from an earlier version, the upgrade will remove any custom metadata sets, and configure the FHIR endpoint with the base FHIR package, either STU3 or R4, depending on what was in use before the upgrade.

If you use custom FHIR search parameters, you must manually migrate them to a FHIR package and apply them to the endpoint before they can be used. The instructions for creating and applying FHIR packages are in the "FHIR Profiles and Adaptations" chapter of *FHIR Support in InterSystems Products*. Using the files that you originally used to used to create the custom metadata set, perform the steps below in the recommended order:

1. Create a custom FHIR package.
2. Import your package or confirm that your package has been imported.
3. Apply the custom package to your endpoint.

Alternatively, you can perform these steps using the FHIR Package API.

# 1.10 Durable %SYS

### Affects Upgrade From Versions:

2019.2 and earlier.

### Conditions:

Environments using a durable %SYS from a 2019.2 or earlier release.

### Ramifications:

In this release, the distribution container has a nonroot default user. This improves the security of your container. Some file ownerships in the host's durable directory must be changed before running this version of InterSystems IRIS. If you do not make these changes, the container will encounter an error starting InterSystems IRIS.

### Instructions:

Please contact your InterSystems sales engineer or the InterSystems Worldwide Response Center for instructions on changing the relevant file ownerships.

# 2

# How to Compile Namespaces

After an upgrade, you must compile the code in each namespace so that it runs under the new version. If the compiler detects any errors, you may need to recompile one or more times for the compiler to resolve all dependencies. Code and namespace compilation require the **%Development_CodeModify:USE** privilege.

After your code compiles successfully, you may want to export any updated classes or routines, in case you need to run the upgrade in any additional environments. Importing these classes or routines during future upgrades allows you to update your code quickly, minimizing downtime.

**Note:** If you are using a manifest as part of your upgrade process, you can compile your code from within the manifest. See the instructions in the Perform Post-Upgrade Tasks section of the Creating and Using an Installation Manifest appendix of this guide.

## 2.1 Compiling Classes

To compile the classes in all namespaces from the Terminal:

```
do $system.OBJ.CompileAllNamespaces("u")
```

To compile the classes in a single namespace from the Terminal:

```
set $namespace = "<namespace>"
do $system.OBJ.CompileAll("u")
```

To compile the classes in the %SYS namespace from the Terminal:

```
set $namespace = "%SYS"
do $system.OBJ.CompileList("%Z*,%z*,Z*,z*,'%ZEN.*,'%ZHS*","up")
```

**Note:** If your namespaces contain mapped classes, include the `/mapped` qualifier in the call to **CompileAllNamespaces()** or **CompileAll()**:

```
do $system.OBJ.CompileAllNamespaces("u /mapped")
do $system.OBJ.CompileAll("u /mapped")
```

# 2.2 Class compiler version utility

To assist customers in determining which class compiler version a class or classes in a namespace have been compiled with, InterSystems provides two assists

- Method – $System.OBJ.CompileInfoClass(<classname>)

  This method returns the version of the class compiler used to compile this <classname> and the datetime the class was compiled

- Query – $System.OBJ.CompileInfo(<sortby>)

  This query generates a report for the current namespace that includes all classes, the version of the compiler used to compile each one, and the datetime each was compiled. The first argument <sortby> may have the following values:

  – 0 – the time the class was compiled

  – 1 – the class name

  – 2 – the version of your product the class was compiled in

# 2.3 Compiling Routines

To compile the routines in all namespaces from the Terminal:

```
do ##Class(%Routine).CompileAllNamespaces()
```

To compile the routines in a single namespace from the Terminal:

```
set $namespace = "<namespace>"
do ##Class(%Routine).CompileAll()
```

To compile the routines in the %SYS namespace from the Terminal:

```
set $namespace = "%SYS"
do ##Class(%Routine).CompileList("%Z*,%z*,Z*,z*,'%ZEN.*,'%ZHS*","up")
```